```
$password = 'weaksauce';
$hash1 = '$2y$10$2alN7H818NP.8BZAoerYxebJIOTsNz4MItmDx1notHgXQwNGxsSCC';
$salt = substr($hash1, 7, 22);
doHash($password, $salt);
```

```
Output:
$2y$10$2alN7H818NP.8BZAoerYxebJIOTsNz4MItmDx1notHgXQwNGxsSCC - took
0.050182104110718 s
```

The precise measurement of time will vary between systems but should be approximately the same (in our case, *0.05* seconds). Finally, let's bump the cost parameter and see what happens:

```
doHash($password, $salt, 11);
doHash($password, $salt, 12);
doHash($password, $salt, 13);
doHash($password, $salt, 14);
```

```
Output:
$2y$11$2alN7H818NP.8BZAoerYxe3upXP1upofGTRoA1STfUjgKO8TWsDqu - took
0.099919080734253 s
$2y$12$2alN7H818NP.8BZAoerYxeeXK7lJ69fxDCEX/znx.Dwe9JJWkye06 - took
0.19856905937195 s
$2y$13$2alN7H818NP.8BZAoerYxeAF8CLsNd6oMOaetBD5lVOLX863ABuHK - took
0.39699101448059 s
$2y$14$2alN7H818NP.8BZAoerYxean/Pk/eNCocT9TKIN23.Jc2sMiHRMVC - took
0.79384803771973 s
```

Notice that as we increment the cost by one, the time taken doubles, and the hash portion of the string changes each time (because the hash is being recomputed on each cycle). The minimum value of cost for bcrypt is `04`, the maximum is `31`, and these are exponential values: a cost of `04` is 2^4, a cost of `31` is 2^{31} – it's possible to have this function run for a very long time indeed. A word of warning: remember rainbow tables, how I warned that they were more than just maps of hashes to strings? What we've just looked at in the example above is a *hash chain*, where the same hash algorithm is applied repeatedly. It is possible to pre-compute such chains, rainbow tables do just this and are therefore able to defeat the workload aspect of costly hash algorithms to a certain extent.